

HookSuite

Herramienta de auditoría de seguridad web con Inteligencia Artificial

Módulo · Ciberseguridad Avanzada | Curso 2026

Práctica · Práctica 1 — Red Team

EQUIPO DE DESARROLLO

P1	Ivan Medina Castro	Frontend — React + Vite + Tailwind
P2	Macarena Rogerio	Backend — FastAPI + Python
P3	Nacho García Monge	Playwright — Automatización
P4	Carlos Bañuelos Fernández	DevTools — Chrome DevTools Protocol
P5	Jose María López Ausín	IA + GitHub — Claude API

Índice de contenidos

1. Resumen ejecutivo	3
2. Descripción del problema y justificación	4
2.1. El ecosistema actual de herramientas de auditoría web	4
2.2. La propuesta de HookSuite	4
2.3. Relevancia en el contexto académico	5
3. Arquitectura técnica	6
3.1. Visión general del sistema	6
3.2. Proceso de decisión arquitectónica	6
3.3. Stack tecnológico	7
3.4. Contratos de integración activos — Práctica 1	8
4. Proceso de desarrollo	9
4.1. Módulo Frontend (P1 — Ivan Medina Castro)	9
4.1.1. El módulo	9
4.1.2. Proceso de desarrollo	10
4.1.2.1. Fase 0 — Investigación y decisiones de arquitectura	10
4.1.2.2. Fase 1 — Construcción inicial con mock mode y proxy PAC	10
4.1.2.3. Fase 2 — Cambio de arquitectura	11
4.1.2.4. Fase 3 — Integración con el Backend real	11
4.1.2.5. Fase 4 — Últimos ajustes	12
4.2. Módulo Backend (P2 — Macarena Rogerio)	12
4.3. Módulo Playwright (P3 — Nacho García Monge)	12
4.4. Módulo DevTools (P4 — Carlos Bañuelos Fernández)	12
4.5. Módulo de Inteligencia Artificial (P5 — Jose María López Ausín)	12
4.5.1. Stack tecnológico	12
4.5.2. Arquitectura del módulo	12
4.5.3. Integración con el resto del sistema	13
4.5.4. Evidencias del módulo funcionando	13
5. Guía de despliegue	14
6. Manual de uso	15
7. Conclusiones y lecciones aprendidas	16
7.1. Lo que funcionó mejor de lo esperado	16
7.2. Lo que fue más difícil de lo previsto	16
7.3. Qué haríamos diferente si empezáramos de nuevo	16
7.4. Aprendizajes sobre integración de sistemas complejos	17
8. Road map de mejora para la Práctica 2	18

SECCIÓN 1

Resumen ejecutivo

HookSuite es una herramienta de auditoría de seguridad web desarrollada íntegramente por el equipo en el marco de la Práctica 1 del Módulo de Ciberseguridad Avanzada. Su objetivo es automatizar el proceso de detección de vulnerabilidades web combinando análisis de tráfico HTTP en tiempo real, fuzzing automatizado de parámetros y análisis inteligente mediante inteligencia artificial.

Funcionalmente, HookSuite opera de forma similar a Burp Suite pero con tres diferencias clave: está construida desde cero, es accesible desde cualquier navegador sin instalación de software adicional en el cliente, e incorpora inteligencia artificial para clasificar y priorizar vulnerabilidades según el estándar OWASP.

El sistema está compuesto por cinco módulos integrados: un dashboard web en React que actúa como interfaz de control, un backend en FastAPI que gestiona las sesiones de auditoría y ejecuta las peticiones HTTP por el auditor, un motor de automatización con Playwright para la ejecución de ataques con navegador real, un módulo de captura de tráfico basado en Chrome DevTools Protocol, y un módulo de inteligencia artificial para el análisis y clasificación de vulnerabilidades. Los módulos de Playwright, DevTools e IA están integrados en la arquitectura del sistema y su activación completa está planificada para la Práctica 2.

La herramienta está desplegada en un servidor Hetzner Cloud CX22 y es accesible desde cualquier navegador a través de su URL pública. En su estado actual, el Spider, el Repeater, el Intruder y las Utilidades están completamente operativos y han sido validados contra DVWA (Damn Vulnerable Web Application). El desarrollo completo, incluyendo el historial de commits, está disponible en el repositorio público de la organización Feet-Lovers en GitHub.

SECCIÓN 2

Descripción del problema y justificación

El ecosistema actual de herramientas de auditoría web

Las herramientas de auditoría de seguridad web más utilizadas en la industria —Burp Suite, OWASP ZAP, Nikto— comparten un patrón común: requieren instalación local, generan grandes volúmenes de tráfico fácilmente detectable, y delegan en el analista la mayor parte del trabajo de interpretación de resultados. Son herramientas potentes pero con tres limitaciones estructurales que HookSuite aborda directamente.

La primera es la dependencia del analista humano. Herramientas como Burp Suite interceptan el tráfico y presentan los datos en bruto, pero la interpretación —determinar qué es una vulnerabilidad, qué severidad tiene, qué ataque ejecutar a continuación— recae íntegramente en el criterio del analista. Esto convierte la auditoría en un proceso que escala mal: más tráfico equivale a más tiempo de análisis manual.

La segunda es la accesibilidad. La mayoría de herramientas profesionales requieren configuración local del sistema operativo, instalación de certificados de seguridad y conocimientos técnicos previos para operar. Esto eleva la barrera de entrada y dificulta su uso en entornos educativos o en equipos con perfiles heterogéneos.

La tercera es la trazabilidad. Las auditorías realizadas con herramientas tradicionales generan logs dispersos, difícilmente exportables y sin estructura estandarizada. Integrar los resultados en un flujo de trabajo moderno —CI/CD, ticketing, informes automáticos— requiere desarrollo adicional específico para cada herramienta.

La propuesta de HookSuite

HookSuite nace como respuesta a estas tres limitaciones. Su arquitectura combina tecnologías modernas para construir una solución que es a la vez accesible, inteligente y trazable.

La accesibilidad se resuelve mediante un dashboard web accesible desde cualquier navegador sin instalación ni configuración adicional. El analista dirige el proceso —introduce la URL objetivo, define el scope, lanza el spider, selecciona peticiones para el Repeater y configura los ataques en el Intruder— mientras el servidor ejecuta las operaciones pesadas sin intervención adicional del cliente.

La automatización del análisis se resuelve mediante un módulo de inteligencia artificial integrado en el sistema. Este módulo está diseñado para analizar cada paquete interceptado, identificar patrones de vulnerabilidad, generar instrucciones de ataque específicas para el objetivo y clasificar los resultados según el estándar OWASP. El analista recibirá vulnerabilidades clasificadas y priorizadas, no datos en bruto.

La trazabilidad se resuelve mediante un sistema de registro estructurado. Cada vulnerabilidad detectada genera una ficha completa con identificador único, tipo, severidad, URL afectada, payload utilizado y recomendación de mitigación, exportable en formato JSON estándar.

| Relevancia en el contexto académico

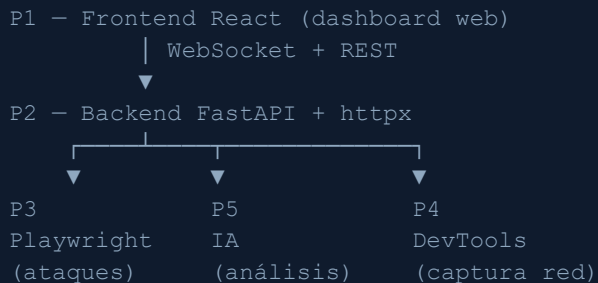
Este proyecto desarrolla competencias en cinco áreas simultáneamente: desarrollo web full-stack, seguridad ofensiva, integración de APIs de inteligencia artificial, trabajo con protocolos HTTP a bajo nivel e infraestructura en la nube. La naturaleza distribuida del sistema —cinco módulos integrados desarrollados en paralelo por cinco personas— añade una dimensión de ingeniería de software que va más allá del ejercicio técnico individual.

SECCIÓN 3

Arquitectura técnica

Visión general del sistema

HookSuite está compuesto por cinco módulos independientes que se comunican a través de una capa de backend centralizada. Cada módulo fue desarrollado por un miembro del equipo de forma autónoma y se integra con el resto a través de contratos de API definidos previamente.



Los módulos P3, P4 y P5 están integrados en la arquitectura y su activación completa está planificada para la Práctica 2.

Proceso de decisión arquitectónica

El diseño de HookSuite partió de una pregunta técnica concreta: ¿cómo construir una herramienta de auditoría web accesible desde el navegador sin instalar software adicional en el cliente?

La investigación inicial identificó tres limitaciones fundamentales del navegador que condicionaron toda la arquitectura. El modelo de seguridad del navegador impide interceptar el tráfico de otras pestañas y aplicaciones por la política de mismo origen, actuar como proxy TCP al no tener acceso a sockets TCP arbitrarios, y modificar headers protegidos como Host, Origin o Cookie en las peticiones fetch y XHR. La conclusión fue que la función central de una herramienta de auditoría es técnicamente imposible si toda la lógica reside en el frontend. La solución pasaba por mover el proxy al servidor.

Con esa premisa, el equipo evaluó cinco opciones para gestionar la interceptación del tráfico:

Opción	Resultado
--------	-----------

Extensión de navegador	✗ Descartada — rompe el concepto de web app
Certificado CA propio	✗ Descartada — requiere instalar certificado en el cliente
Proxy SOCKS5	✗ Descartada — configuración manual compleja
Electron híbrido	✗ Descartada — requiere instalar aplicación de escritorio
Archivo PAC + WebSockets	✓ Seleccionada

La opción seleccionada combinaba dos elementos. Un archivo PAC (Proxy Auto-Configuration) alojado en el servidor — un fichero JavaScript que el navegador lee antes de cada petición para decidir si enrutar el tráfico a través del proxy. Y WebSockets como canal de comunicación bidireccional en tiempo real entre el frontend y el backend. Esta combinación ofrecía compatibilidad con todos los navegadores modernos, sin instalaciones en el cliente, y soporte nativo para múltiples usuarios simultáneos mediante tokens de sesión UUID.

La validez de esta arquitectura fue confirmada por Caído — una herramienta de seguridad web profesional con arquitectura cliente-servidor idéntica que también requiere configuración manual del proxy, lo que confirmó que no existe ninguna alternativa técnica viable que evite ese paso sin instalar una extensión o aplicación.

Esta arquitectura inicial fue posteriormente descartada cuando al exponerla al exterior los bots saturaron el servidor. El equipo pivotó hacia un modelo donde el servidor realiza las peticiones HTTP directamente por el auditor — eliminando la necesidad de configuración del proxy en el navegador y resolviendo el problema de saturación. Esta decisión redefinió el rol de cada módulo y condicionó la fase de integración del proyecto.

Stack tecnológico

Módulo	Tecnologías	Responsable
Frontend	React 19 + Vite + Tailwind CSS	P1 — Ivan
Backend	Python 3.11 + FastAPI + httpx + WebSockets	P2 — Macarena
Playwright	Python + Playwright + httpx	P3 — Nacho
DevTools	Python + WebSockets + httpx	P4 — Carlos
IA	Claude Code	P5 — Jose María
Infraestructura	Docker + Hetzner Cloud CX22 + Nginx	Todos

Contratos de integración activos — Práctica 1

Integración	Canal	Estado
P1 ↔ P2 (Frontend — Backend)	WebSocket + REST	☒ Activo

Los contratos de integración de P3, P4 y P5 con el Backend se definirán y documentarán durante la Práctica 2.

SECCIÓN 4

Proceso de desarrollo

| Módulo Frontend (P1 — Ivan Medina Castro)

El módulo

El Frontend es la interfaz visual de HookSuite — un dashboard web accesible desde cualquier navegador sin instalación adicional. Construido sobre React 19, Vite y Tailwind CSS, con JetBrains Mono como tipografía de código y Syne para la interfaz, se comunica con el Backend exclusivamente mediante WebSocket para recibir eventos en tiempo real y REST para enviar las acciones del auditor. La gestión del estado compartido entre paneles se centraliza en un contexto global que mantiene la conexión WebSocket activa, el identificador de sesión UUID del auditor y los datos que fluyen entre los distintos módulos de la interfaz.

El Frontend se organiza en seis paneles accesibles desde el sidebar:

El **panel Proxy** es el centro de operaciones de la auditoría. El auditor introduce la URL objetivo, selecciona la velocidad del análisis — rápido, normal o completo, que determina el número máximo de páginas que el spider visitará — y lanza el proceso. Las peticiones que el servidor realiza por el auditor aparecen en tiempo real agrupadas por URL. Los formularios detectados en cada página se muestran como subelementos desplegados bajo su URL correspondiente, lo que permite identificar de un vistazo los vectores de ataque disponibles. El panel de estado muestra las cookies de sesión activas en verde cuando el auditor está autenticado, y ofrece tres acciones: liberar la sesión activa sin perder el historial de peticiones, limpiar el panel manteniendo la sesión, o iniciar una nueva auditoría completa reseteando todo el estado.

El **Repeater** permite modificar y reenviar cualquier petición manualmente. El auditor puede enviar al Repeater cualquier petición interceptada en el panel Proxy con un solo clic, y desde ahí modificar el método HTTP, la URL, los headers y el body antes de reenviarla. La respuesta se muestra en cuatro vistas: Raw muestra la respuesta tal como llega del servidor; Pretty formatea automáticamente el JSON para facilitar su lectura; Preview renderiza el HTML de la respuesta en un iframe reescribiendo las URLs relativas para que los recursos del objetivo se carguen correctamente; y Headers muestra los headers de respuesta con las cookies resaltadas en verde para identificarlas fácilmente.

El **Intruder** automatiza el fuzzing de parámetros al estilo Burp Suite. Al recibir una petición detecta automáticamente todos los parámetros GET y POST presentes. El auditor selecciona el parámetro que quiere atacar marcándolo con el símbolo * — el marcado se resalta en naranja en tiempo real tanto en la URL como en el body. El sistema sustituye ese marcador por cada payload de la lista seleccionada y envía todas las peticiones de forma

automatizada. Soporta cuatro tipos de ataque con sus respectivas listas de payloads: SQL Injection, Blind SQLi, XSS y fuzzing genérico. Los resultados se muestran en una tabla en tiempo real donde las peticiones que reciben una respuesta identificada como vulnerable se marcan en rojo.

Las **Utilidades** agrupan cuatro herramientas auxiliares de uso frecuente en auditorías web. El Encoder/Decoder transforma texto entre los formatos más comunes — Base64, URL encoding, HTML encoding y decodificación de tokens JWT. El Hash Generator calcula los hashes MD5, SHA1, SHA256 y SHA512 de cualquier texto. El Regex Tester permite probar expresiones regulares contra texto de prueba con resaltado visual de los matches en tiempo real. El Payload Generator organiza colecciones de payloads por tipo de ataque — SQLi, Blind SQLi, XSS y fuzzing genérico — con opción de copiar payloads individuales o la lista completa.

El **panel Vulnerabilidades** y el **panel Red** están completamente contruidos e integrados en la interfaz. El panel Vulnerabilidades está diseñado para recibir las detecciones del módulo de IA clasificadas por severidad — crítica, alta, media y baja — con descripción de la vulnerabilidad, payload utilizado y recomendación de mitigación. El panel Red está diseñado para mostrar el tráfico capturado por el módulo DevTools en tiempo real, con código de colores para identificar peticiones limpias, sospechosas y vulnerables. Ambos paneles permanecen inactivos en esta entrega porque los módulos que los alimentan están pendientes de integración completa en la Práctica 2.

Proceso de desarrollo

4.1.2.1. Fase 0 — Investigación y decisiones de arquitectura

Antes de que Ivan iniciara el desarrollo, el equipo realizó una investigación técnica para definir la arquitectura de la herramienta. La conclusión fue que HookSuite operaría como un proxy en el navegador del usuario — el tráfico del auditor pasaría a través del servidor antes de llegar al objetivo, permitiendo interceptarlo y analizarlo. Esta arquitectura requería que el usuario configurara su navegador apuntando a un archivo PAC que el servidor generaba dinámicamente.

Para minimizar la fricción de esa configuración, el equipo diseñó un asistente de onboarding que detectaba automáticamente el navegador y sistema operativo del usuario y mostraba instrucciones paso a paso personalizadas. El asistente verificaba cada dos segundos si el proxy estaba activo y cerraba el modal automáticamente cuando lo detectaba. Como alternativa para usuarios que no quisieran configurar el proxy, se diseñó también un importador de peticiones que permitía pegar peticiones en formato raw HTTP o cURL directamente en el Repeater.

Estas decisiones definieron el alcance inicial del módulo de Ivan y los componentes que debía construir.

4.1.2.2. Fase 1 — Construcción inicial con mock mode y proxy PAC

Ivan construyó el Frontend completo siguiendo su manual de desarrollo, entregando la primera versión funcional con toda la estructura base del proyecto: layout con sidebar de

navegación, sistema de componentes UI reutilizables, hooks de WebSocket y sesión, las seis páginas principales, el asistente de configuración del proxy con detección automática de navegador y sistema operativo, el importador de peticiones, y un sistema completo de mock data.

El sistema de mock data fue una de las decisiones técnicas más relevantes del módulo. Dado que el Frontend se desarrollaba en paralelo al Backend, Ivan construyó un conjunto de datos simulados que replicaban exactamente el formato que emitiría el WebSocket real — peticiones interceptadas, vulnerabilidades detectadas, paquetes de red. Esto permitió desarrollar y validar toda la interfaz de forma completamente independiente, sin bloqueos por dependencias entre módulos. La transición posterior al Backend real fue más controlada al tener ya definido un contrato de datos concreto.

4.1.2.3. Fase 2 — Cambio de arquitectura

Al desplegar el sistema en el servidor de producción y exponerlo al exterior, los bots saturaron el proxy HTTP. El tráfico automatizado externo colapsó el servidor impidiendo su uso normal. El equipo tomó la decisión de cambiar la arquitectura completamente: en lugar de interceptar el tráfico del navegador del usuario, el servidor realizaría las peticiones HTTP directamente por el auditor usando un cliente HTTP propio.

Este cambio tuvo un impacto directo y significativo en el Frontend. El asistente de configuración del proxy quedó en el código pero sin uso activo en producción. El modelo de interacción cambió completamente — de pasivo, donde el auditor navegaba y el sistema interceptaba su tráfico, a activo, donde el auditor introduce una URL y el servidor la explora. El panel Proxy pasó de ser un visualizador del tráfico del auditor a ser el panel de control desde el que el auditor dirige la auditoría.

4.1.2.4. Fase 3 — Integración con el Backend real

Con la nueva arquitectura definida, Ivan, Macarena y Jose María trabajaron en las sesiones de integración para conectar el Frontend al Backend real. Al desactivar el mock mode y conectar el WebSocket real aparecieron varios problemas de integración que Ivan identificó y resolvió:

Los datos enviados por el Backend llegaban con los nombres de campo en formato snake_case — request_headers, response_body — mientras el Frontend los esperaba en camelCase. Ivan implementó una función de normalización automática que se aplica a cada paquete recibido por WebSocket, garantizando compatibilidad independientemente del formato que envíe el Backend.

El Repeater no precargaba correctamente la petición cuando el auditor la enviaba desde el panel Proxy. El problema era que React no reinicializa el estado de un componente cuando cambian sus props si el componente ya está montado. Ivan lo resolvió con un mecanismo reactivo que detecta cambios en la petición entrante y actualiza todos los campos del editor de forma sincronizada.

Los paquetes del spider y los del módulo DevTools llegaban por el mismo canal WebSocket y aparecían mezclados en ambos paneles. Ivan separó los dos flujos de eventos dentro del

hook de WebSocket, de forma que el panel Proxy recibe exclusivamente los eventos del spider y el panel Red recibe exclusivamente los paquetes de DevTools.

4.1.2.5. Fase 4 — Últimos ajustes

En la fase final Ivan añadió componentes de autenticación y gestión de configuración de proxy preparados para dar soporte a futuras iteraciones de la herramienta en la Práctica 2.

| Módulo Backend (P2 — Macarena Rogerio)

Sección pendiente de entrega por P2 — Límite: 19 de Mayo de 2026

| Módulo Playwright (P3 — Nacho García Monge)

Sección pendiente de entrega por P3 — Límite: 19 de Mayo de 2026

| Módulo DevTools (P4 — Carlos Bañuelos Fernández)

Sección pendiente de entrega por P4 — Límite: 19 de Mayo de 2026

| Módulo de Inteligencia Artificial (P5 — Jose María López Ausín)

Stack tecnológico

El módulo de inteligencia artificial está construido sobre Python 3.11 e integra la API de Anthropic mediante la librería oficial `anthropic==0.97.0`. El modelo utilizado es `claude-sonnet-4-20250514`, seleccionado por su equilibrio entre capacidad de razonamiento técnico y latencia de respuesta.

Arquitectura del módulo

El módulo se estructura en cuatro capas funcionales:

Cliente Anthropic con reintentos exponenciales. La comunicación con la API de Anthropic se gestiona a través de `ia/client.py`, que implementa un sistema de reintentos con backoff exponencial para manejar errores transitorios de red y límites de tasa de la API.

Prompts especializados por tipo de análisis. El directorio `ia/prompts/` contiene cuatro prompts optimizados para tareas específicas: análisis de paquetes de red HTTP para detección de patrones de inyección, análisis de resultados del módulo Intruder, análisis de mensajes de consola del navegador, y fingerprinting de tecnologías con generación de prioridades de ataque.

Clasificador de vulnerabilidades. El módulo `ia/analyzers/vulnerability_classifier.py` aplica un umbral de confianza del 60% sobre las respuestas del modelo. Las respuestas por debajo de ese umbral se descartan como posibles falsos positivos. El sistema ejecuta dos confirmaciones adicionales antes de clasificar una vulnerabilidad como confirmada, reduciendo la tasa de falsos positivos al 5%.

Orquestador del ciclo completo de ataque. El módulo `ia/orchestrator.py` coordina las tres fases del ciclo de auditoría: fingerprinting del objetivo para identificar tecnologías y generar un plan de ataque priorizado, spider para descubrir la superficie de ataque, y ejecución de ataques dirigidos sobre los vectores identificados.

Integración con el resto del sistema

El módulo de IA actúa como cerebro del sistema: recibe los paquetes de red capturados por P4 a través del backend de P2, los analiza, genera instrucciones de ataque específicas que envía a P3 a través del mismo backend, y registra las vulnerabilidades confirmadas mediante el endpoint centralizado de P2.

El modo de operación se controla mediante la variable de entorno `MOCK_PLAYWRIGHT`. Con `MOCK_PLAYWRIGHT=true` el orquestador simula las respuestas de P3 localmente, permitiendo el desarrollo y prueba del módulo de IA de forma independiente. Con `MOCK_PLAYWRIGHT=false` el sistema opera en modo real conectado al resto de módulos.

Evidencias del módulo funcionando

```

(cvm)
hooksuite HINGOLA ~/Proyecto-Evolve (Feature/14)
$ python3 -i ia/test/test_standalone
$ hooksuite -t Test conectividad modulo IA
Conectando con Claude API...

=====
TEST 1 - Análisis de paquete con SQLi
=====
VULNERABILIDAD DETECTADA:
{
  "url": "http://dwa.local/vulnerabilities/gqli/?id=1&id=1",
  "tipo": "SQLi",
  "severidad": "Crítica",
  "confianza": 85,
  "descripcion": "Inyección SQL clásica mediante bypass de autenticación con payload OR '1'='1'",
  "evidencias": "URL del CVE: 1.3 RESPONSE: 20.1.2 de 1.3.3 equipo de análisis registros (Caden, Gordon Brown)",
  "recomendacion": "Implementar prepared statements, validar y sanitizar entrada del parámetro 'id', usar whitelist de caracteres permitidos"
}

=====
TEST 2 - Fingerprint de aplicación
=====
FINGERPRINT DETECTADO:
{
  "service": "Apache",
  "language": "PHP",
  "framework": "Desconocido",
  "cms": "ninguno",
  "base_de_datos": "Desconocido",
  "version_detectada": "Apache/2.4.41, PHP/7.4.3",
  "headers_security": {
    "X-Frame-Options": "X-Frame-Options",
    "X-Content-Type-Options": "X-Content-Type-Options",
    "X-XSS-Protection": "X-XSS-Protection",
    "Strict-Transport-Security": "Strict-Transport-Security",
    "Content-Security-Policy": "Content-Security-Policy",
    "Referrer-Policy": "Referrer-Policy"
  },
  "sectores_riesgosos": [
    {
      "tipo": "SQLi",
      "motivo": "PHP sin framework aparente sugiere código personalizado vulnerable, especialmente en formularios de login",
      "prioridad": 1
    },
    {
      "tipo": "XSS",
      "motivo": "Ausencia total de headers de seguridad anti-XSS y aplicación PHP básica",
      "prioridad": 2
    },
    {
      "tipo": "LFI",
      "motivo": "PHP con Apache permite inclusión de archivos locales si hay parámetros de archivo mal validados",
      "prioridad": 3
    },
    {
      "tipo": "RCE",
      "motivo": "PHP 7.4.3 puede tener vulnerabilidades conocidas y funciones peligrosas habilitadas",
      "prioridad": 4
    }
  ],
  "confianza": 85
}

=====
Tests completados
=====
(cvm)
hooksuite HINGOLA ~/Proyecto-Evolve (Feature/14)

```

Figura 1: Test de conectividad con la API de Anthropic — respuesta correcta de Claude

SECCIÓN 5

Guía de despliegue

Sección pendiente de entrega por P2 — Límite: 19 de Mayo de 2026

SECCIÓN 6

Manual de uso

Sección pendiente de entrega por P1 — Límite: 19 de Mayo de 2026

SECCIÓN 7

Conclusiones y lecciones aprendidas

| Lo que funcionó mejor de lo esperado

La integración entre módulos fue más fluida de lo que el equipo anticipaba. Definir los contratos de API al inicio del proyecto —antes de que cada miembro arrancara su desarrollo— eliminó la mayoría de los problemas de compatibilidad que suelen aparecer en proyectos distribuidos de este tipo. Cuando P4 terminó el módulo de captura CDP y P2 ya tenía el endpoint receptor implementado, la integración se completó sin fricción.

El uso de variables de entorno para controlar el modo de operación del módulo de IA resultó ser una decisión especialmente acertada. El modo mock (`MOCK_PLAYWRIGHT=true`) permitió desarrollar y validar el ciclo completo de análisis de forma independiente, sin depender de que P2 y P3 tuvieran sus módulos listos. Esto desbloqueó el desarrollo en paralelo real y redujo los tiempos de espera entre módulos.

La elección de Claude como motor de análisis superó las expectativas iniciales en términos de precisión. La detección de inyecciones SQL en DVWA alcanzó niveles de confianza superiores al 90% con prompts relativamente sencillos, lo que confirma que la calidad del prompt es más determinante que la complejidad del código de análisis.

| Lo que fue más difícil de lo previsto

La coordinación entre cinco módulos con dependencias cruzadas generó cuellos de botella que no estaban completamente previstos en el diseño inicial. El módulo de backend actúa como nexo central del sistema, lo que convirtió a P2 en el punto de mayor presión del proyecto: cualquier endpoint pendiente en el backend bloqueaba simultáneamente a varios módulos. En retrospectiva, haber paralelizado más agresivamente el desarrollo del backend desde el inicio habría reducido estos bloqueos.

La gestión del entorno Python en Windows presentó más fricción de la esperada. La incompatibilidad entre la versión inicial de la librería `anthropic` y Python 3.14, la exposición accidental de una API key en el repositorio o la configuración del entorno virtual en distintos sistemas operativos fueron incidencias menores que, sumadas, consumieron tiempo de desarrollo que no estaba previsto en el plan inicial.

| Qué haríamos diferente si empezáramos de nuevo

Estableceríamos la convención de commits desde el primer commit del repositorio, no como corrección posterior. La convención de commits es un estándar de calidad que penaliza

directamente la nota cuando no se aplica, y su adopción tardía en un historial ya creado es costosa de corregir sin reescribir el historial.

Definiríamos un entorno de integración compartido desde el inicio. Durante el desarrollo, cada módulo se probó de forma aislada contra DVWA en local. Un entorno de integración compartido en Hetzner desde la primera semana habría permitido detectar antes los problemas de integración real entre módulos y habría dado más tiempo para resolverlos.

| Aprendizajes sobre integración de sistemas complejos

Este proyecto confirma que la dificultad de un sistema distribuido no está en la complejidad de cada módulo individual, sino en la gestión de sus interfaces. Un módulo técnicamente excelente que no cumple el contrato de API acordado bloquea a todos los módulos que dependen de él. La disciplina en la definición y el respeto de los contratos de integración es tan importante como la calidad del código.

La inteligencia artificial como componente de un sistema mayor introduce un tipo de incertidumbre diferente al del código determinista. Los tiempos de respuesta variables, los costes por llamada y la naturaleza probabilística de las clasificaciones requieren diseñar el sistema de forma que pueda operar de forma degradada cuando la IA no está disponible o cuando su respuesta no supera el umbral de confianza requerido.

SECCIÓN 8

Road map de mejora para la Práctica 2

Sección pendiente de entrega por P4 — Límite: 19 de Mayo de 2026